Chapter 8: Discussion, System Evaluation, and Future Work

By: KHAN, Muhammad Mustafa

8.1 Critical Evaluation of System Security Implementation

Throughout Phase 2, our team (Group 7, Topic 15) conducted a thorough security assessment and engineering review of our web application. Our main goal was to test and strengthen our application against common web threats from the OWASP Top 10 list, focusing specifically on **A05: Security Misconfiguration** and **A10: Server-Side Request Forgery / Security Logging and Monitoring Failures**.

When we performed our initial baseline security testing, we discovered several critical configuration issues before applying our fixes. Under **A05: Security Misconfiguration**, our Flask Web Application Server was running with `FLASK_DEBUG=1` enabled in both testing and staging environments. Whenever an unhandled error or database issue occurred, the Werkzeug interactive debugger showed full stack traces, environment variables, and internal memory details directly in the browser. This could allow an unauthenticated attacker to see our software setup, find hidden security keys, and potentially run unauthorized code on our server using the debug tools.

Additionally, when we analyzed our HTTP response headers, we noticed that key security protections were missing. Our app was serving responses without enforcing content-type rules (`X-Content-Type-Options: nosniff`), leaving users exposed to MIME-sniffing attacks. We were also missing frame protections (`X-Frame-Options: DENY`), which opened up clickjacking risks where an attacker could embed our site into a malicious frame. Finally, we didn't have HTTP Strict Transport Security (`HSTS`) enabled, leaving network traffic vulnerable to protocol downgrade attacks.

Under **A10: Security Logging and Monitoring Failures**, we found that our application logging was disorganized and incomplete. Important features like user login, password resets, admin role changes, and API calls weren't creating proper audit logs. When someone tried an invalid login or attempted a brute-force attack, nothing was saved in our server logs. Without structured logs, we had no way of knowing if someone was

scanning our app or trying to gain unauthorized access, making incident response almost impossible.

8.2 Security Remediation, Engineering, and Architecture Analysis

To fix these issues, we implemented a clear step-by-step engineering plan across our server setup and codebase.

1. 8.2.1 A05: Security Misconfiguration Fixes

2. **Hardening the Environment and Error Handling:** We updated our configuration so that `FLASK_DEBUG=0` is strictly enforced across all deployment environments. We added custom error handlers at the application level to handle standard HTTP 4xx and 5xx error codes. Instead of showing full stack traces to users, our application now displays simple error pages while writing the actual error details to our internal server logs.
3. **Adding HTTP Security Headers:** We updated our backend middleware using WSGI wrappers so that necessary security headers are automatically added to every outgoing HTTP response:

- `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload`: Forces browsers to use secure HTTPS connections for all traffic over the next year.
- `X-Frame-Options: DENY`: Blocks browsers from loading our pages inside `<frame>`, `<iframe>`, `<embed>`, or `<object>` tags to stop clickjacking attacks.
- `X-Content-Type-Options: nosniff`: Prevents browsers from guessing file types, protecting against malicious script execution.
- `Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'`: Restricts script and layout resources to trusted local origins, protecting against cross-site scripting (XSS).

4. **Hiding Server Information:** We cleaned up and removed server headers (`Server: Werkzeug/...`) so attackers can't easily identify what tools and versions we are running behind the scenes.

8.2.2 A10: Security Logging and Monitoring Infrastructure

1. **Setting Up Centralized JSON Logging:** We refactored our code to replace basic `print` statements with Python's built-in `logging` library configured to export structured JSON logs. Each log entry automatically records key

information: ISO timestamps, severity level (`INFO`, `WARNING`, `ERROR`, `CRITICAL`), client IP address, requested URL, HTTP method, user ID, and the outcome of the request.

2. **Adding Audit Triggers:** We added explicit log entries across all security-sensitive routes:

- **Login & Logout Events:** We log every login attempt. Failed logins are logged at the `WARNING` level with the target username and client IP address so we can spot brute-force attempts.
- **Access Control Failures:** Whenever a user tries to access a page they don't have permission for, we record an `ERROR` level log with their user details and the path they tried to open.

3. **Cleaning and Redacting Sensitive Data:** We wrote custom log filters to scrub confidential information before writing to disk. Our regex filters scan log entries for keywords like `password`, `token`, `secret`, and `credit_card`, automatically replacing sensitive values with `[REDACTED]`.

8.3 Fail-Open vs. Fail-Closed Strategy

A major design goal in Phase 2 was deciding how our system should handle unexpected errors during permission checks.

- **Risks of Fail-Open Systems:** In poorly designed applications, when an authentication system encounters an unexpected failure (like a database timeout or a missing identity service), it defaults to letting the request through. This "fail-open" behavior means that when something breaks, unauthorized users might accidentally gain access to restricted parts of the system.
- **Our Fail-Closed Implementation:** To make sure our system stays secure at all times, we designed a strict "fail-closed" model across all our route handlers, middleware, and database layers. We wrapped our access control logic in `try-catch-finally` blocks. If an unexpected error, database disconnection, or permission issue happens:
    1. Our security checks immediately revoke active user sessions and cancel temporary tokens.
    2. The app writes a `CRITICAL` severity entry to our security logs.
    3. The request is immediately stopped before any sensitive database queries or actions take place.

4. The user sees a standard `HTTP 403 Forbidden` or `HTTP 500 Internal Server Error` message without revealing any backend details.

8.4 How We Used Agile Project Management

We ran Phase 2 using an **Agile (Scrum)** approach broken into two-week sprints. Working iteratively helped us stay on track, split up tasks effectively, and adapt whenever we hit technical roadblocks.

1. **Sprint Planning and Sprint Goals:** At the start of each sprint, we broke down our project requirements into smaller user stories and estimated their effort together. We organized our sprints around finding vulnerabilities, testing exploits, implementing backend fixes, and writing our final reports. Our regular check-ins helped us pivot quickly—for example, when we had to adjust how our Flask app injected WSGI headers.
2. **Dividing Up Tasks:** We assigned tasks based on each group member's technical strengths. Some of us worked on infrastructure hardening and backend logic, while others built test scripts and put together our documentation and reports.
3. **Team Communication and Reviews:** We held regular standup discussions and reviewed each other's work through pull requests before merging code changes. This peer-review process helped us catch configuration errors early and keep our code clean throughout the project.

8.5 Future Work and Next Steps

While we successfully fixed our main security issues in Phase 2, we have planned a few key improvements for future development cycles:

1. **Automated DevSecOps Pipeline:** We want to add static analysis tools (like Bandit and SonarQube) and dynamic security tools (OWASP ZAP) directly into our GitHub Actions CI/CD workflows. This will automatically scan our code during pull requests to make sure new changes don't introduce security misconfigurations or logging gaps.
2. **SIEM Integration and Real-Time Alerts:** We plan to connect our JSON logs to a centralized monitoring system like the Elastic Stack (ELK) or Splunk. Setting up

continuous log aggregation will allow us to build dashboards and send instant alert notifications when suspicious activity (like a spike in failed logins) occurs.

3. **Stronger Access Control (RBAC/ABAC):** We aim to expand our access controls from simple role checks to fine-grained Attribute-Based Access Control (ABAC), making sure user data remains fully isolated across different user accounts.